



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ
FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES
INGENIERIA DE SOFTWARE



CURSO:

IMPLEMENTACIÓN DE BASE DE DATOS II

TEMA:

PROYECTO FINAL

FACILITADOR:

HENRY LEZCANO

ESTUDIANTES:

MIRIAM ANGULO 8-1025-1615

DAVID PIMENTEL 8-1010-750

DIEGO JAEN 7-718-1410

ELBIS VALLEJOS 9-766-202

CARLOS DELGADO E-8-204947

GRUPO: 1SF131

Resumen

El proyecto Tourlstm tiene como objetivo principal el diseño e implementación de una base de datos sólida y escalable para una plataforma web dedicada a promover el turismo en Panamá. Esta aplicación busca mejorar la experiencia de usuarios locales e internacionales, centralizando información actualizada sobre destinos, planes turísticos y actividades. Además, ofrece una solución eficiente para que las empresas turísticas puedan difundir sus servicios de forma ordenada y visible.

El diseño del sistema partió de un modelo conceptual y lógico relacional que fue transformado en un modelo físico implementado en Oracle. Se aplicaron principios de normalización hasta tercera forma normal, definición de claves primarias y foráneas, e integridad referencial para garantizar la consistencia de los datos. Se desarrollaron entidades clave como usuarios, empresas, destinos turísticos, planes, favoritos, comentarios e historial de interacción. A nivel de programación almacenada, se implementaron procedimientos, funciones, triggers y cursores en PL/SQL que automatizan operaciones como la validación de contenido, gestión de favoritos y generación de recomendaciones personalizadas, incorporando también estructuras de control y manejo de excepciones.

Se priorizó la usabilidad, seguridad y rendimiento del sistema mediante autenticación, encriptación, respaldo de datos y una interfaz web responsiva. En conjunto, Tourlstm demuestra cómo una base de datos bien diseñada puede ser el núcleo de una solución digital funcional, innovadora y con potencial real en el mercado turístico, posicionándose como un proyecto ideal para incluir en un portafolio profesional.

Índice

Introducción	3
Misión de la Base de Datos	5
Objetivos de la misión de la Base de Datos	5
Definición del Proyecto.....	6
Alcance o Límites:.....	6
Análisis de Requerimientos.....	6
Modelado de la Base de Datos	9
Modelo Lógico Relacional	10
Modelo Físico según el sistema de gestión (Oracle)	12
Implementación de PL/SQL.	19
Conclusión	38
Bibliografía/Referencias	40

Introducción

En un mundo cada vez más conectado digitalmente, la industria del turismo ha tenido que adaptarse a nuevas formas de interacción con los usuarios. Las personas ya no dependen únicamente de agencias físicas o guías impresas para planificar sus viajes; ahora buscan plataformas digitales que les brinden información confiable, recomendaciones personalizadas y experiencias interactivas. Esta realidad presenta una oportunidad para desarrollar soluciones tecnológicas que no solo respondan a las necesidades de los turistas modernos, sino que también apoyen a las empresas del sector en la promoción de sus servicios.

Bajo esta visión, nace Tourlstm, un proyecto académico desarrollado como parte del curso Implementación de Base de Datos II. Su objetivo principal es construir una base de datos sólida, escalable y bien estructurada que sirva como la base funcional de una plataforma web centrada en la promoción del turismo en Panamá. Esta plataforma está diseñada para ofrecer a los usuarios una experiencia completa: desde explorar destinos turísticos por categoría o ubicación, hasta guardar sus favoritos y recibir sugerencias personalizadas según sus intereses. Al mismo tiempo, brinda a las empresas turísticas un espacio digital donde puedan registrar y publicar planes turísticos de manera organizada y segura.

A lo largo del proyecto, se abordan las diferentes fases de diseño e implementación de una base de datos real: desde la creación del modelo conceptual hasta su traducción a un modelo lógico y físico implementado en Oracle. También se desarrollan e integran múltiples objetos de programación almacenada, como procedimientos, funciones, triggers y cursores, los cuales permiten automatizar tareas, aplicar reglas de negocio y mantener la integridad de los datos.

Este informe documenta el proceso completo de desarrollo del proyecto, desde sus fundamentos conceptuales hasta su implementación técnica, demostrando cómo una base de datos bien diseñada puede convertirse en el corazón de una solución tecnológica robusta, útil y con potencial real de aplicación en el mercado actual.

Tourlstm

Misión de la Base de Datos

Crear una base de datos robusta y escalable que centralice y gestione información turística actualizada y confiable de Panamá, facilitando la promoción de destinos, planes turísticos y recomendaciones personalizadas para usuarios y empresas.

Objetivos de la misión de la Base de Datos

- Reunir en un solo sistema todos los datos sobre destinos turísticos, actividades y evaluaciones de usuarios en un único sistema.
- Brindar una plataforma confiable para que las empresas promociones sus servicios.
- Proteger la información almacenada mediante seguridad y disponer de una estructura tecnológica que permita el crecimiento del sistema conforme aumente los usuarios y las cantidades de datos gestionados.
- Fomentar la participación de los usuarios en la actualización del contenido relacionado con destinos y actividades turísticas, garantizando la transparencia y veracidad de la información proporcionada.
- Implementar un sistema de registro y publicación de destinos turísticos que permita a los usuarios autorizados proponer nuevos lugares de interés cultural, natural, urbano y tropical en las diferentes provincias y distritos de Panamá.
- Desarrollar un sistema de autenticación y autorización robusto que diferencie entre roles de usuario (cliente y administrador) con sus respectivos permisos y funcionalidades
- Crear mecanismos de seguridad para proteger la información sensible de usuarios y empresas mediante el almacenamiento seguro de contraseñas.

Definición del Proyecto

Ámbito:

Tourlstmo se dirige tanto a turistas nacionales como internacionales que deseen explorar y conocer Panamá. La aplicación provee:

- Promover los principales destinos turísticos de Panamá para visitantes tanto locales como extranjeros.
- Brindar a las empresas turísticas una plataforma donde puedan publicar y gestionar sus planes.
- Facilitar la búsqueda de destinos según intereses, regiones o experiencias del usuario.
- Permitir la interacción del usuario guardando favoritos de destino.
- Integrar un mapa interactivo donde se puedan ubicar los destinos.
- Funcionalidad para guardar destinos de interés, y acceder a ellos desde el perfil.

Alcance o Límites:

El proyecto contempla el desarrollo de una aplicación web completamente responsiva, compatible con todo tipo de dispositivos, incluidos móviles y tabletas. Esta aplicación incluirá funcionalidades modernas como scroll dinámico y filtros personalizables para facilitar la búsqueda de destinos turísticos. Los usuarios tendrán la posibilidad de sugerir nuevos destinos, agregando información relevante, la cual será evaluada y validada por un administrador antes de su publicación para garantizar la calidad y evitar duplicidad.

En cuanto a sus límites, la aplicación se enfocará inicialmente en destinos y servicios turísticos dentro del territorio panameño. En esta primera etapa, los planes turísticos publicados solo funcionarán como referencia informativa; es decir, los usuarios no podrán realizar reservas ni efectuar pagos directamente desde la plataforma.

Análisis de Requerimientos

Requerimientos Funcionales:

1. Visualización de Destinos Turísticos

El sistema debe permitir que cualquier usuario pueda visualizar una lista de destinos turísticos disponibles. Esta lista debe estar bien organizada y mostrar información esencial de cada destino, como su nombre, la categoría a la que pertenece (por ejemplo, playa, montaña, ciudad histórica, etc.) y una imagen representativa que ayude a identificar el lugar de manera visual. Esta funcionalidad es clave para que el usuario tenga una primera impresión atractiva y pueda explorar las opciones disponibles.

2. Visualización Detallada de un Destino

Cuando el usuario selecciona un destino específico, el sistema debe redirigirlo a una página de detalle. En esta página se debe mostrar una galería de imágenes del destino, una descripción completa que explique sus principales características o atractivos, una calificación promedio basada en las opiniones de otros usuarios y un mapa interactivo que permita ubicar el destino geográficamente. Esta vista detallada busca ofrecer toda la información necesaria para que el usuario pueda evaluar si desea visitarlo.

3. Gestión de Planes Turísticos

Los usuarios registrados en el sistema podrán crear y publicar sus propios planes turísticos. Al momento de registrar un plan, deberán incluir información como el nombre del plan, una descripción clara, la duración del viaje, el precio estimado, las fechas en las que estará disponible, imágenes relacionadas con el plan y los datos de contacto del usuario que lo publica. Esta funcionalidad fomenta la participación activa de los usuarios y permite compartir experiencias personalizadas.

4. Gestión de Usuarios y Seguridad

El sistema debe ofrecer funciones básicas de gestión de usuarios, como el registro de nuevas cuentas, el inicio de sesión mediante correo electrónico y contraseña. Para garantizar la privacidad y seguridad de la información, se deben implementar mecanismos de validación de datos y control de acceso.

5. Funcionalidad de Favoritos

Los usuarios tendrán la posibilidad de marcar destinos como favoritos. Esto les permitirá guardar aquellos lugares que más les interesen y acceder a ellos rápidamente desde su perfil. Esta opción mejora la experiencia del usuario al ofrecer una forma personalizada de organizar los destinos que desea visitar.

6. Panel de Administración

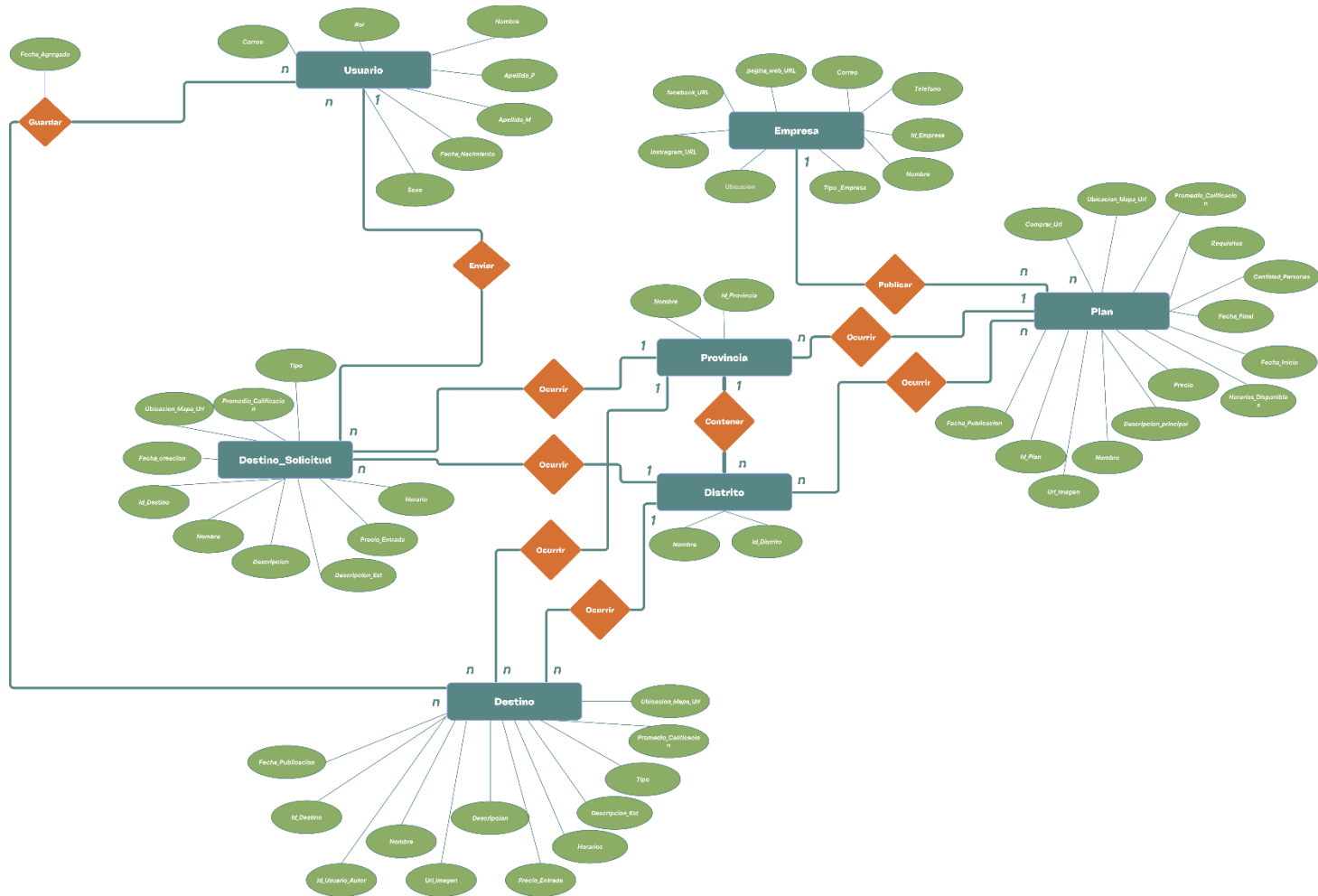
El sistema debe contar con un panel exclusivo para administradores, desde el cual se pueda gestionar y aprobar el contenido generado por los usuarios y empresas. Los administradores también podrán validar nuevos destinos propuestos, asegurándose de que la información sea real y adecuada. Además, este panel debe mostrar estadísticas generales del sistema, como la cantidad de destinos registrados, planes publicados, usuarios activos y otras métricas relevantes para el control y mejora del sistema.

Requerimientos No Funcionales:

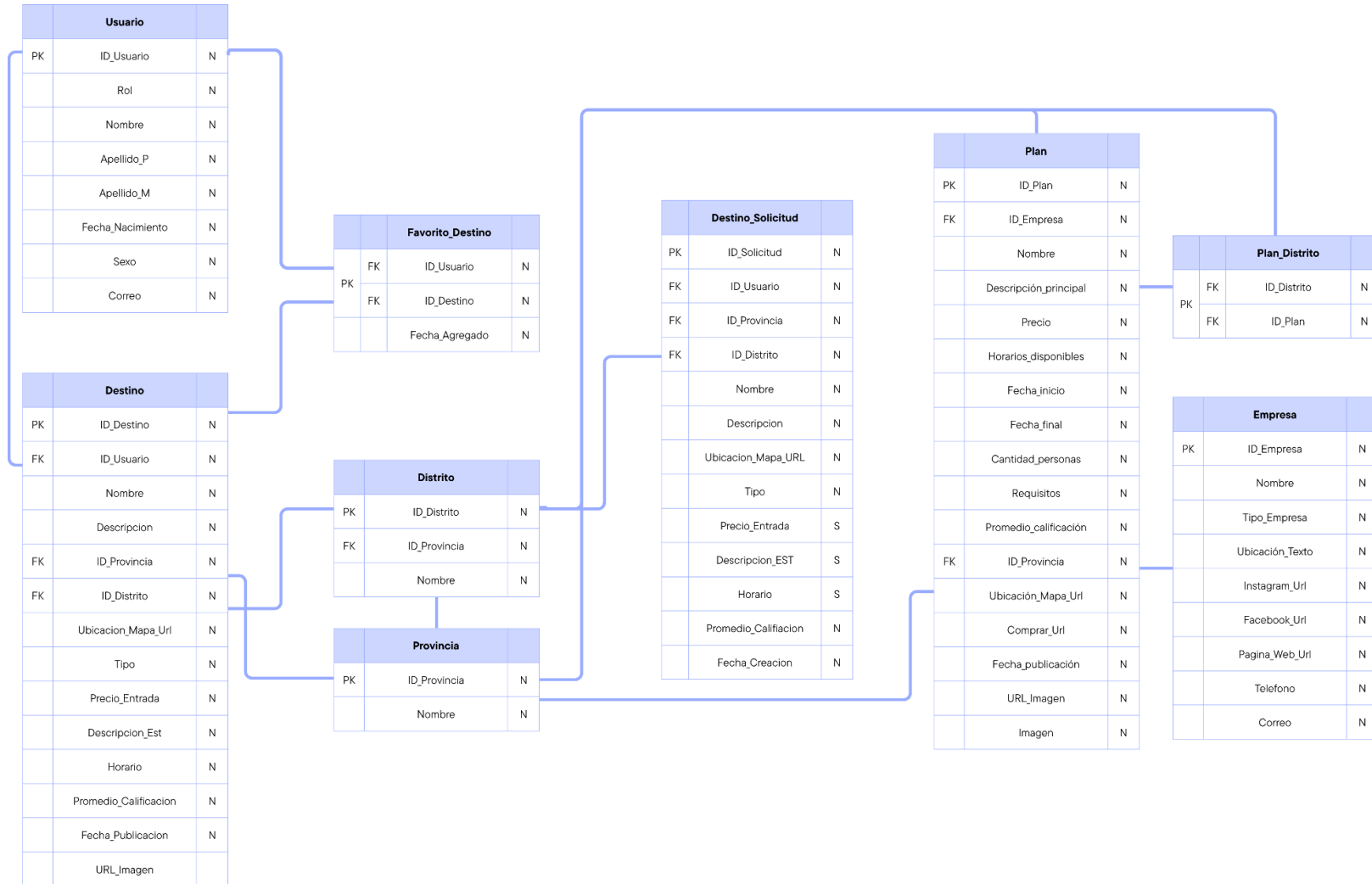
- **Escalabilidad:** Arquitectura modular y orientada a microservicios (si se extiende) para soportar crecimiento en usuarios y volumen de datos.
- **Seguridad:** Implementación de encriptación, autenticación robusta y cumplimiento de normativas (ej. GDPR).
- **Rendimiento:** Optimización de consultas y uso de índices en la base de datos para garantizar tiempos de respuesta rápidos.
- **Disponibilidad y Fiabilidad:** Uso de sistemas de respaldo y alta disponibilidad para minimizar tiempos de inactividad.
- **Usabilidad:** Interfaz amigable y diseño responsivo, accesible en diferentes dispositivos y para usuarios con discapacidades.

Modelado de la Base de Datos

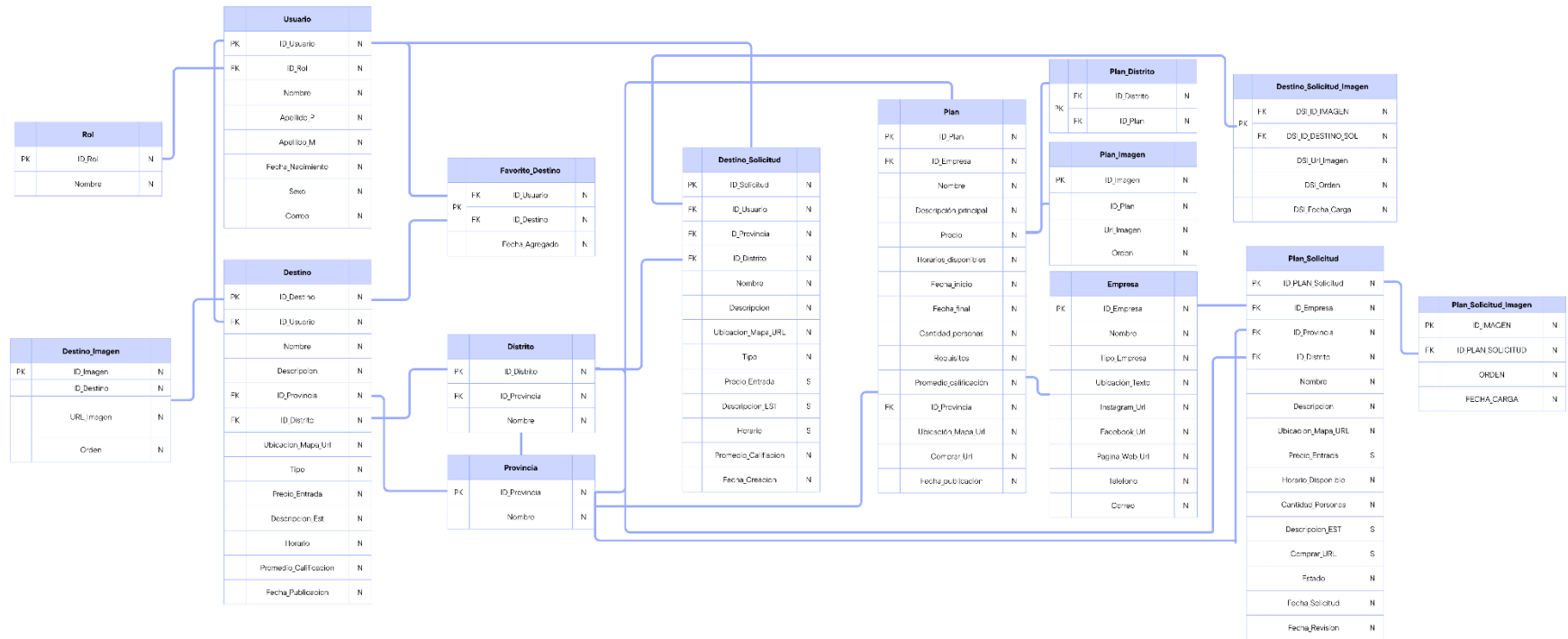
-Modelo Conceptual E/R



Modelo Lógico Relacional



Modelo Lógico Relacional Normalizado



Modelo Físico según el sistema de gestión (Oracle)

```
SQL> */
SQL>
SQL> CREATE SEQUENCE seq_rol          START WITH 1 INCREMENT BY 1 NOCACHE;
Sequence created.

SQL> CREATE SEQUENCE seq_usuario      START WITH 1 INCREMENT BY 1 NOCACHE;
Sequence created.

SQL> CREATE SEQUENCE seq_empresa      START WITH 1 INCREMENT BY 1 NOCACHE;
Sequence created.

SQL> CREATE SEQUENCE seq_provincia    START WITH 1 INCREMENT BY 1 NOCACHE;
Sequence created.

SQL> CREATE SEQUENCE seq_distrito     START WITH 1 INCREMENT BY 1 NOCACHE;
Sequence created.

SQL> CREATE SEQUENCE seq_destino_sol  START WITH 1 INCREMENT BY 1 NOCACHE;
Sequence created.

SQL> CREATE SEQUENCE seq_destino      START WITH 1 INCREMENT BY 1 NOCACHE;
Sequence created.

SQL> CREATE SEQUENCE seq_destino_img  START WITH 1 INCREMENT BY 1 NOCACHE;
Sequence created.

SQL> CREATE SEQUENCE seq_plan         START WITH 1 INCREMENT BY 1 NOCACHE;
Sequence created.

SQL> CREATE SEQUENCE seq_plan_img     START WITH 1 INCREMENT BY 1 NOCACHE;
Sequence created.

SQL> CREATE SEQUENCE seq_comentario   START WITH 1 INCREMENT BY 1 NOCACHE;
Sequence created.

SQL> CREATE SEQUENCE seq_plan_solicitud START WITH 1 INCREMENT BY 1 NOCACHE;
Sequence created.

SQL> CREATE SEQUENCE seq_plan_sol_img  START WITH 1 INCREMENT BY 1 NOCACHE;
Sequence created.

SQL> CREATE SEQUENCE seq_destino_sol_img START WITH 1 INCREMENT BY 1 NOCACHE;
Sequence created.
```

```

SQL>
SQL>
SQL>
SQL> CREATE TABLE ROL (
  2   ROL_ID_ROL      NUMBER PRIMARY KEY,
  3   ROL_NOMBRE      VARCHAR2(20) UNIQUE NOT NULL
  4 );

Table created.

SQL>
SQL> INSERT INTO ROL VALUES (1,'CLIENTE');

1 row created.

SQL> INSERT INTO ROL VALUES (2,'ADMIN');

1 row created.

SQL>
SQL>
SQL> CREATE TABLE USUARIO (
  2   USU_ID_USUARIO  NUMBER PRIMARY KEY,
  3   USU_ID_ROL      NUMBER NOT NULL,
  4   USU_NOMBRE      VARCHAR2(60) NOT NULL,
  5   USU_APELLIDO_P  VARCHAR2(60) NOT NULL,
  6   USU_APELLIDO_M  VARCHAR2(60),
  7   USU_FECHA_NACIMIENTO DATE,
  8   USU_SEXO        CHAR(1),
  9   USU_CORREO       VARCHAR2(120) NOT NULL,
 10   USU_CONTRASENA   VARCHAR2(100) NOT NULL,
 11   USU_IMAGEN_PERFIL VARCHAR2(255),
 12   CONSTRAINT UK_USU_CORREO UNIQUE (USU_CORREO),
 13   CONSTRAINT fk_usu_rol FOREIGN KEY (USU_ID_ROL)
 14   REFERENCES ROL (ROL_ID_ROL)
 15 );

Table created.

SQL>
SQL>
SQL> CREATE TABLE EMPRESA (
  2   EMP_ID_EMPRESA  NUMBER PRIMARY KEY,
  3   EMP_NOMBRE      VARCHAR2(120) NOT NULL,
  4   EMP_TIPO_EMPRESA VARCHAR2(60),
  5   EMP_UBICACION_TEXTO VARCHAR2(255),
  6   EMP_INSTAGRAM_URL VARCHAR2(255),
  7   EMP_FACEBOOK_URL VARCHAR2(255),
  8   EMP_PAGINA_WEB_URL VARCHAR2(255),
  9   EMP_TELEFONO     VARCHAR2(30),
 10   EMP_CORREO       VARCHAR2(120),
 11   EMP_CONTRASENA   VARCHAR2(100) NOT NULL, -- Nueva columna para la contraseña
 12   CONSTRAINT UK_EMP_CORREO UNIQUE (EMP_CORREO)
 13 );

Table created.

```

```

SQL> CREATE TABLE PROVINCIA (
  2   PRO_ID_PROVINCIA      NUMBER PRIMARY KEY,
  3   PRO_NOMBRE           VARCHAR2(60) UNIQUE NOT NULL
  4 );

Table created.

SQL>
SQL> CREATE TABLE DISTRITO (
  2   DIS_ID_DISTRITO       NUMBER PRIMARY KEY,
  3   DIS_ID_PROVINCIA      NUMBER NOT NULL,
  4   DIS_NOMBRE           VARCHAR2(60) NOT NULL,
  5   CONSTRAINT fk_dis_prov FOREIGN KEY (DIS_ID_PROVINCIA)
  6       REFERENCES PROVINCIA (PRO_ID_PROVINCIA),
  7   CONSTRAINT uq_dis_nombre UNIQUE (DIS_ID_PROVINCIA, DIS_NOMBRE)
  8 );

Table created.

SQL>
SQL>
SQL>      /* --- DESTINO_SOLICITUD --- */
SQL> CREATE TABLE DESTINO_SOLICITUD (
  2   DSO_ID_DESTINO        NUMBER PRIMARY KEY,
  3   DSO_ID_USUARIO_AUTOR  NUMBER NOT NULL,
  4   DSO_NOMBRE            VARCHAR2(120) NOT NULL,
  5   DSO_DESCRIPCION       VARCHAR2(400),
  6   DSO_ID_PROVINCIA      NUMBER,
  7   DSO_ID_DISTRITO      NUMBER,
  8   DSO_UBICACION_MAPA_URL VARCHAR2(1000),
  9   DSO_TIPO              VARCHAR2(60),
 10   DSO_PRECIO_ENTRADA     NUMBER(10,2),
 11   DSO_DESCRIPCION_EST    VARCHAR2(20) DEFAULT 'PENDIENTE',
 12   DSO_HORARIO           VARCHAR2(255),
 13   DSO_PROMEDIO_CALIFICACION NUMBER(3,2),
 14   DSO_FECHA_CREACION    TIMESTAMP DEFAULT SYSTIMESTAMP,
 15   CONSTRAINT fk_dso_usu FOREIGN KEY (DSO_ID_USUARIO_AUTOR)
 16       REFERENCES USUARIO (USU_ID_USUARIO),
 17   CONSTRAINT fk_dso_pro FOREIGN KEY (DSO_ID_PROVINCIA)
 18       REFERENCES PROVINCIA (PRO_ID_PROVINCIA),
 19   CONSTRAINT fk_dso_dis FOREIGN KEY (DSO_ID_DISTRITO)
 20       REFERENCES DISTRITO (DIS_ID_DISTRITO),
 21   CONSTRAINT chk_dso_estado CHECK (DSO_DESCRIPCION_EST IN ('PENDIENTE', 'APROBADO', 'RECHAZADO'))
 22 );

Table created.

```

```

SQL>      /* --- DESTINO_SOLICITUD --- */
SQL> CREATE TABLE DESTINO_SOLICITUD (
  2   DSO_ID_DESTINO          NUMBER PRIMARY KEY,
  3   DSO_ID_USUARIO_AUTOR    NUMBER NOT NULL,
  4   DSO_NOMBRE              VARCHAR2(120) NOT NULL,
  5   DSO_DESCRIPCION         VARCHAR2(400),
  6   DSO_ID_PROVINCIA        NUMBER,
  7   DSO_ID_DISTRITO         NUMBER,
  8   DSO_UBICACION_MAPA_URL  VARCHAR2(1000),
  9   DSO_TIPO                VARCHAR2(60),
10   DSO_PRECIO_ENTRADA       NUMBER(10,2),
11   DSO_DESCRIPCION_EST      VARCHAR2(20) DEFAULT 'PENDIENTE',
12   DSO_HORARIO              VARCHAR2(255),
13   DSO_PROMEDIO_CALIFICACION NUMBER(3,2),
14   DSO_FECHA_CREACION      TIMESTAMP DEFAULT SYSTIMESTAMP,
15   CONSTRAINT fk_dso_usu FOREIGN KEY (DSO_ID_USUARIO_AUTOR)
16       REFERENCES USUARIO (USU_ID_USUARIO),
17   CONSTRAINT fk_dso_pro FOREIGN KEY (DSO_ID_PROVINCIA)
18       REFERENCES PROVINCIA (PRO_ID_PROVINCIA),
19   CONSTRAINT fk_dso_dis FOREIGN KEY (DSO_ID_DISTRITO)
20       REFERENCES DISTRITO (DIS_ID_DISTRITO),
21   CONSTRAINT chk_dso_estado CHECK (DSO_DESCRIPCION_EST IN ('PENDIENTE', 'APROBADO', 'RECHAZADO'))
22 );

```

Table created.

```

SQL>
SQL>
SQL>      /* --- DESTINO --- */
SQL> CREATE TABLE DESTINO (
  2   DES_ID_DESTINO          NUMBER PRIMARY KEY,
  3   DES_ID_USUARIO_AUTOR    NUMBER NOT NULL,
  4   DES_NOMBRE              VARCHAR2(120) NOT NULL,
  5   DES_DESCRIPCION         VARCHAR2(400),
  6   DES_ID_PROVINCIA        NUMBER,
  7   DES_ID_DISTRITO         NUMBER,
  8   DES_UBICACION_MAPA_URL  VARCHAR2(1000),
  9   DES_TIPO                VARCHAR2(60),
10   DES_PRECIO_ENTRADA       NUMBER(10,2),
11   DES_DESCRIPCION_EST      VARCHAR2(20),
12   DES_HORARIO              VARCHAR2(255),
13   DES_PROMEDIO_CALIFICACION NUMBER(3,2),
14   DES_FECHA_PUBLICACION    TIMESTAMP DEFAULT SYSTIMESTAMP,
15   CONSTRAINT fk_des_usu FOREIGN KEY (DES_ID_USUARIO_AUTOR)
16       REFERENCES USUARIO (USU_ID_USUARIO),
17   CONSTRAINT fk_des_pro FOREIGN KEY (DES_ID_PROVINCIA)
18       REFERENCES PROVINCIA (PRO_ID_PROVINCIA),
19   CONSTRAINT fk_des_dis FOREIGN KEY (DES_ID_DISTRITO)
20       REFERENCES DISTRITO (DIS_ID_DISTRITO),
21   CONSTRAINT UK_DES_NOMBRE UNIQUE (DES_NOMBRE)
22 );

```

Table created.

```

SQL> /* --- DESTINO_IMAGEN (prefijo DIM_) --- */
SQL> CREATE TABLE DESTINO_IMAGEN (
  2   DIM_ID_IMAGEN          NUMBER PRIMARY KEY,
  3   DIM_ID_DESTINO         NUMBER NOT NULL,
  4   DIM_URL_IMAGEN         VARCHAR2(300) NOT NULL,
  5   DIM_ORDEN              NUMBER(5),
  6   CONSTRAINT fk_dim_des FOREIGN KEY (DIM_ID_DESTINO)
  7                           REFERENCES DESTINO (DES_ID_DESTINO)
  8                           ON DELETE CASCADE
  9 );

```

Table created.

```

SQL>
SQL>
SQL>
SQL>
SQL> /* --- PLAN (prefijo PLA_) --- */
SQL> CREATE TABLE PLAN (
  2   PLA_ID_PLAN            NUMBER PRIMARY KEY,
  3   PLA_ID_EMPRESA         NUMBER NOT NULL,
  4   PLA_NOMBRE             VARCHAR2(120) NOT NULL,
  5   PLA_DESCRIPCION_PRINCIPAL VARCHAR2(400),
  6   PLA_PRECIO             NUMBER(10,2),
  7   PLA_HORARIOS_DISPONIBLES VARCHAR2(255),
  8   PLA_FECHA_INICIO       DATE,
  9   PLA_FECHA_FINAL        DATE,
 10   PLA_CANTIDAD_PERSONAS  NUMBER,
 11   PLA_REQUISITOS         VARCHAR2(400),
 12   PLA_PROMEDIO_CALIFICACION NUMBER(3,2),
 13   PLA_ID_PROVINCIA       NUMBER,
 14   PLA_ID_DISTRITO        NUMBER,
 15   PLA_UBICACION_MAPA_URL VARCHAR2(1000),
 16   PLA_COMPRAR_URL        VARCHAR2(1000),
 17   PLA_FECHA_PUBLICACION  TIMESTAMP DEFAULT SYSTIMESTAMP,
 18   CONSTRAINT fk_pla_emp FOREIGN KEY (PLA_ID_EMPRESA)
 19                           REFERENCES EMPRESA (EMP_ID_EMPRESA),
 20   CONSTRAINT fk_pla_pro FOREIGN KEY (PLA_ID_PROVINCIA)
 21                           REFERENCES PROVINCIA (PRO_ID_PROVINCIA),
 22   CONSTRAINT fk_pla_dis FOREIGN KEY (PLA_ID_DISTRITO)
 23                           REFERENCES DISTRITO (DIS_ID_DISTRITO)
 24 );

```

Table created.

```

SQL>
SQL> /* --- PLAN_IMAGEN (prefijo PIM_) - */
SQL> CREATE TABLE PLAN_IMAGEN (
  2   PIM_ID_IMAGEN          NUMBER PRIMARY KEY,
  3   PIM_ID_PLAN            NUMBER NOT NULL,
  4   PIM_URL_IMAGEN         VARCHAR2(255) NOT NULL,
  5   PIM_ORDEN              NUMBER(5),
  6   CONSTRAINT fk_pim_pla FOREIGN KEY (PIM_ID_PLAN)
  7                           REFERENCES PLAN (PLA_ID_PLAN)
  8                           ON DELETE CASCADE
  9 );

```

Table created.


```

SQL> /* --- FAVORITO_DESTINO (prefijo FAV_) */
SQL> CREATE TABLE FAVORITO_DESTINO (
  2   FAV_ID_USUARIO      NUMBER NOT NULL,
  3   FAV_ID_DESTINO      NUMBER NOT NULL,
  4   FAV_FECHA_AGREGADO  TIMESTAMP DEFAULT SYSTIMESTAMP,
  5   PRIMARY KEY (FAV_ID_USUARIO, FAV_ID_DESTINO),
  6   CONSTRAINT fk_fav_usu FOREIGN KEY (FAV_ID_USUARIO)
  7       REFERENCES USUARIO (USU_ID_USUARIO)
  8       ON DELETE CASCADE,
  9   CONSTRAINT fk_fav_des FOREIGN KEY (FAV_ID_DESTINO)
 10       REFERENCES DESTINO (DES_ID_DESTINO)
 11       ON DELETE CASCADE
 12 );

Table created.

SQL>
SQL> CREATE TABLE PLAN_DISTrito (
  2   PD_ID_PLAN          NUMBER NOT NULL,
  3   PD_ID_DISTrito      NUMBER NOT NULL,
  4   CONSTRAINT PK_PLAN_DISTrito
  5       PRIMARY KEY (PD_ID_PLAN, PD_ID_DISTrito),
  6   CONSTRAINT FK_PD_PLAN
  7       FOREIGN KEY (PD_ID_PLAN)
  8       REFERENCES PLAN (PLA_ID_PLAN),
  9   CONSTRAINT FK_PD_DISTrito
 10       FOREIGN KEY (PD_ID_DISTrito)
 11       REFERENCES DISTrito (DIS_ID_DISTrito)
 12 );

Table created.

SQL>
SQL> /* ===== TABLA PLAN_SOLICITUD ===== */
SQL> -- Esta tabla almacenará las solicitudes de planes antes de ser aprobadas
SQL> CREATE TABLE PLAN_SOLICITUD (
  2   PSO_ID_PLAN_SOLICITUD  NUMBER PRIMARY KEY,
  3   PSO_ID_EMPRESA          NUMBER NOT NULL,
  4   PSO_NOMBRE              VARCHAR2(120) NOT NULL,
  5   PSO_DESCRIPCION_PRINCIPAL VARCHAR2(400),
  6   PSO_PRECIO              NUMBER(10,2),
  7   PSO_HORARIOS_DISPONIBLES VARCHAR2(255),
  8   PSO_FECHA_INICIO        DATE,
  9   PSO_FECHA_FINAL         DATE,
 10   PSO_CANTIDAD_PERSONAS   NUMBER,
 11   PSO_REQUISITOS          VARCHAR2(400),
 12   PSO_ID_PROVINCIA         NUMBER,
 13   PSO_ID_DISTrito         NUMBER,
 14   PSO_UBICACION_MAPA_URL  VARCHAR2(1000),
 15   PSO_COMPRAR_URL         VARCHAR2(1000),
 16   PSO_ESTADO              VARCHAR2(20) DEFAULT 'PENDIENTE',
 17   PSO_FECHA_SOLICITUD     TIMESTAMP DEFAULT SYSTIMESTAMP,
 18   PSO_FECHA_REVISION      TIMESTAMP,
 19   -- Claves foráneas
 20   CONSTRAINT fk_pso_emp FOREIGN KEY (PSO_ID_EMPRESA)
 21       REFERENCES EMPRESA (EMP_ID_EMPRESA),
 22   CONSTRAINT fk_pso_pro FOREIGN KEY (PSO_ID_PROVINCIA)
 23       REFERENCES PROVINCIA (PRO_ID_PROVINCIA),
 24   CONSTRAINT fk_pso_dis FOREIGN KEY (PSO_ID_DISTrito)
 25       REFERENCES DISTrito (DIS_ID_DISTrito),
 26   -- Constraint para validar estados
 27   CONSTRAINT chk_pso_estado CHECK (PSO_ESTADO IN ('PENDIENTE', 'APROBADO', 'RECHAZADO', 'EN_REVISION'))
 28 );

```

```

SQL> /* ===== TABLA PLAN_SOLICITUD_IMAGEN ===== */
SQL> -- Imágenes asociadas a las solicitudes de planes
SQL> CREATE TABLE PLAN_SOLICITUD_IMAGEN (
  2     PSI_ID_IMAGEN          NUMBER PRIMARY KEY,
  3     PSI_ID_PLAN_SOLICITUD  NUMBER NOT NULL,
  4     PSI_URL_IMAGEN         VARCHAR2(255) NOT NULL,
  5     PSI_ORDEN              NUMBER(5),
  6     PSI_FECHA_CARGA        TIMESTAMP DEFAULT SYSTIMESTAMP,
  7     -- Clave foránea
  8     CONSTRAINT fk_psi_pso FOREIGN KEY (PSI_ID_PLAN_SOLICITUD)
  9                           REFERENCES PLAN_SOLICITUD (PSO_ID_PLAN_SOLICITUD)
10                           ON DELETE CASCADE
11 );

```

Table created.

```

SQL>
SQL>
SQL> /* ===== TABLA DESTINO_SOLICITUD_IMAGEN ===== */
SQL> -- Imágenes asociadas a las solicitudes de destinos
SQL> CREATE TABLE DESTINO_SOLICITUD_IMAGEN (
  2     DSI_ID_IMAGEN          NUMBER PRIMARY KEY,
  3     DSI_ID_DESTINO_SOL      NUMBER NOT NULL,
  4     DSI_URL_IMAGEN         VARCHAR2(255) NOT NULL,
  5     DSI_ORDEN              NUMBER(5),
  6     DSI_FECHA_CARGA        TIMESTAMP DEFAULT SYSTIMESTAMP,
  7     -- Clave foránea
  8     CONSTRAINT fk_dsi_dso FOREIGN KEY (DSI_ID_DESTINO_SOL)
  9                           REFERENCES DESTINO_SOLICITUD (DSO_ID_DESTINO)
10                           ON DELETE CASCADE
11 );

```

Table created.

Implementación de PL/SQL.

1. Procedimiento registrar_empresa:

```
SQL> CREATE OR REPLACE PROCEDURE registrar_empresa(  
2     p_nombre          IN VARCHAR2,  
3     p_tipo_empresa    IN VARCHAR2,  
4     p_ubicacion       IN VARCHAR2,  
5     p_telefono        IN VARCHAR2,  
6     p_correo          IN VARCHAR2,  
7     p_contrasena     IN VARCHAR2,  
8     p_instagram       IN VARCHAR2 DEFAULT NULL,  
9     p_facebook        IN VARCHAR2 DEFAULT NULL,  
10    p_pagina_web       IN VARCHAR2 DEFAULT NULL,  
11    p_id_empresa      OUT NUMBER  
12 )  
13 IS  
14 BEGIN  
15     SELECT seq_empresa.NEXTVAL INTO p_id_empresa FROM DUAL;  
16  
17     INSERT INTO EMPRESA (  
18         EMP_ID_EMPRESA, EMP_NOMBRE, EMP_TIPO_EMPRESA, EMP_UBICACION_TEXTO,  
19         EMP_TELEFONO, EMP_CORREO, EMP_CONTRASENA,  
20         EMP_INSTAGRAM_URL, EMP_FACEBOOK_URL, EMP_PAGINA_WEB_URL  
21     ) VALUES (  
22         p_id_empresa, p_nombre, p_tipo_empresa, p_ubicacion,  
23         p_telefono, p_correo, p_contrasena,  
24         p_instagram, p_facebook, p_pagina_web  
25     );  
26  
27     COMMIT;  
28 EXCEPTION  
29     WHEN DUP_VAL_ON_INDEX THEN  
30         RAISE_APPLICATION_ERROR(-20001, 'El correo ya está registrado');  
31     WHEN OTHERS THEN  
32         ROLLBACK;  
33         RAISE_APPLICATION_ERROR(-20002, 'Error al registrar empresa: ' || SQLERRM);  
34 END;  
35 /_
```

2. Procedimiento registrar_usuario:

```
SQL> CREATE OR REPLACE PROCEDURE registrar_usuario(  
 2     p_nombre           IN VARCHAR2,  
 3     p_apellido_p       IN VARCHAR2,  
 4     p_apellido_m       IN VARCHAR2,  
 5     p_fecha_nacimiento IN DATE,  
 6     p_sexo             IN CHAR,  
 7     p_correo           IN VARCHAR2,  
 8     p_contrasena      IN VARCHAR2, -- Nuevo parámetro  
 9     p_imagen_perfil    IN VARCHAR2 DEFAULT NULL,  
10     p_id_usuario      OUT NUMBER  
11 ) IS  
12 BEGIN  
13     -- Asignar ID de usuario usando la secuencia  
14     SELECT seq_usuario.NEXTVAL INTO p_id_usuario FROM DUAL;  
15  
16     -- Insertar el nuevo usuario (rol por defecto: CLIENTE - 1)  
17     INSERT INTO USUARIO (  
18         USU_ID_USUARIO,  
19         USU_ID_ROL,  
20         USU_NOMBRE,  
21         USU_APELLIDO_P,  
22         USU_APELLIDO_M,  
23         USU_FECHA_NACIMIENTO,  
24         USU_SEXO,  
25         USU_CORREO,  
26         USU_CONTRASENA, -- Nuevo campo  
27         USU_IMAGEN_PERFIL  
28     ) VALUES (  
29         p_id_usuario,  
30         1, -- CLIENTE por defecto  
31         p_nombre,  
32         p_apellido_p,  
33         p_apellido_m,  
34         p_fecha_nacimiento,  
35         p_sexo,  
36         p_correo,  
37         p_contrasena, -- Valor recibido  
38         p_imagen_perfil  
39     );  
40  
41     COMMIT;  
42 EXCEPTION  
43     WHEN DUP_VAL_ON_INDEX THEN  
44         RAISE_APPLICATION_ERROR(-20001, 'El correo ya está registrado');  
45     WHEN OTHERS THEN  
46         ROLLBACK;  
47         RAISE_APPLICATION_ERROR(-20002, 'Error al registrar usuario: ' || SQLERRM);  
48 END registrar_usuario;  
49 /
```

Procedure created.

3. Procedimiento solicitud_destino

```
SQL> CREATE OR REPLACE PROCEDURE solicitud_destino(  
2   p_id_usuario      IN NUMBER,  
3   p_nombre          IN VARCHAR2,  
4   p_descripcion     IN CLOB,  
5   p_id_provincia    IN NUMBER,  
6   p_id_distrito     IN NUMBER,  
7   p_ubicacion_mapa  IN VARCHAR2 DEFAULT NULL,  
8   p_tipo            IN VARCHAR2,  
9   p_precio_entrada  IN NUMBER DEFAULT NULL,  
10  p_descripcion_est  IN VARCHAR2 DEFAULT 'PENDIENTE',  
11  p_horario          IN VARCHAR2 DEFAULT NULL,  
12  p_id_destino_sol   OUT NUMBER  
13 ) IS  
14 -- Variable para verificar si el usuario existe  
15 v_usuario_count NUMBER;  
16  
17 -- Cursor para listar solicitudes existentes del usuario (ejemplo de cursor)  
18 CURSOR c_solicitudes IS  
19   SELECT DSO_ID_DESTINO, DSO_NOMBRE  
20   FROM DESTINO_SOLICITUD  
21   WHERE DSO_ID_USUARIO_AUTOR = p_id_usuario;  
22  
23 BEGIN  
24   -- Validar que el usuario existe  
25   SELECT COUNT(*) INTO v_usuario_count  
26   FROM USUARIO  
27   WHERE USU_ID_USUARIO = p_id_usuario;  
28  
29   IF v_usuario_count = 0 THEN  
30     RAISE_APPLICATION_ERROR(-20005, 'El usuario no existe');  
31   END IF;  
32  
33   -- Mostrar solicitudes previas del usuario (ejemplo de cursor + loop)  
34   FOR solicitud IN c_solicitudes LOOP  
35     -- Aquí podrías hacer alguna lógica adicional, por ejemplo, logging o validación  
36     DBMS_OUTPUT.PUT_LINE('Solicitud existente ID=' || solicitud.DSO_ID_DESTINO || ', Nombre=' || solicitud.DSO_NOMBRE);  
37   END LOOP;  
38  
39   -- Asignar ID para la nueva solicitud  
40   SELECT seq_destino_sol.NEXTVAL INTO p_id_destino_sol FROM DUAL;  
41  
42   -- Insertar la solicitud de destino  
43   INSERT INTO DESTINO_SOLICITUD (  
44     DSO_ID_DESTINO,  
45     DSO_ID_USUARIO_AUTOR,  
46     DSO_NOMBRE,  
47     DSO_DESCRIPCION,  
48     DSO_ID_PROVINCIA,  
49     DSO_ID_DISTrito,  
50     DSO_UBICACION_MAPA_URL,  
51     DSO_TIPO,  
52     DSO_PRECIO_ENTRADA,  
53     DSO_DESCRIPCION_EST,  
54     DSO_HORARIO  
55   ) VALUES (  
56     p_id_destino_sol,  
57     p_id_usuario,  
58     p_nombre,  
59     p_descripcion,  
60     p_id_provincia,  
61     p_id_distrito,  
62     p_ubicacion_mapa,  
63     p_tipo,  
64     p_precio_entrada,  
65     p_descripcion_est,  
66     p_horario  
67   );  
68  
69   COMMIT;  
70  
71 EXCEPTION  
72   WHEN NO_DATA_FOUND THEN  
73     -- Excepción por si algo falla en búsqueda  
74     ROLLBACK;  
75     RAISE_APPLICATION_ERROR(-20010, 'Error: dato no encontrado');  
76   WHEN DUP_VAL_ON_INDEX THEN  
77     ROLLBACK;  
78     RAISE_APPLICATION_ERROR(-20011, 'Error: ya existe una solicitud con datos duplicados');  
79   WHEN OTHERS THEN  
80     ROLLBACK;  
81     RAISE_APPLICATION_ERROR(-20002, 'Error al solicitar destino: ' || SQLERRM);  
82 END solicitud_destino;  
83 /
```

4. Procedimiento agregar_imagen_destino_sol:

```
SQL> CREATE OR REPLACE PROCEDURE agregar_imagen_destino_sol(
  2     p_id_destino_sol IN NUMBER,
  3     p_url_imagen     IN VARCHAR2,
  4     p_orden          IN NUMBER DEFAULT NULL,
  5     p_id_imagen      OUT NUMBER
  6 ) IS
  7     v_destino_sol_exist NUMBER;
  8     v_orden            NUMBER;
  9 BEGIN
 10     -- Verificar que la solicitud de destino existe
 11     SELECT COUNT(*) INTO v_destino_sol_exist
 12     FROM DESTINO_SOLICITUD
 13     WHERE DSO_ID_DESTINO = p_id_destino_sol;
 14
 15     IF v_destino_sol_exist = 0 THEN
 16         RAISE_APPLICATION_ERROR(-20007, 'La solicitud de destino no existe');
 17     END IF;
 18
 19     -- Asignar orden para la imagen, si no se especificó
 20     IF p_orden IS NULL THEN
 21         SELECT NVL(MAX(DSI_ORDEN), 0) + 1 INTO v_orden
 22         FROM DESTINO_SOLICITUD_IMAGEN
 23         WHERE DSI_ID_DESTINO_SOL = p_id_destino_sol;
 24     ELSE
 25         v_orden := p_orden;
 26     END IF;
 27
 28     -- Obtener nuevo ID para la imagen usando secuencia
 29     SELECT seq_destino_sol_img.NEXTVAL INTO p_id_imagen FROM DUAL;
 30
 31     -- Insertar la nueva imagen
 32     INSERT INTO DESTINO_SOLICITUD_IMAGEN (
 33         DSI_ID_IMAGEN,
 34         DSI_ID_DESTINO_SOL,
 35         DSI_URL_IMAGEN,
 36         DSI_ORDEN
 37     ) VALUES (
 38         p_id_imagen,
 39         p_id_destino_sol,
 40         p_url_imagen,
 41         v_orden
 42     );
 43
 44     COMMIT;
 45
 46 EXCEPTION
 47     WHEN OTHERS THEN
 48         ROLLBACK;
 49         RAISE_APPLICATION_ERROR(-20008, 'Error al agregar imagen: ' || SQLERRM);
 50 END agregar_imagen_destino_sol;
 51 /
```

Procedure created.

5. Procedimiento aprobar_destino:

```
SQL> --Procedimiento para aprobar una solicitud de destino
SQL> CREATE OR REPLACE PROCEDURE aprobar_destino(
  2   p_id_destino_sol IN NUMBER,
  3   p_id_destino OUT NUMBER
  4 ) IS
  5   v_datos_destino DESTINO_SOLICITUD%ROWTYPE;
  6 BEGIN
  7   -- Obtener los datos de la solicitud
  8   BEGIN
  9     SELECT * INTO v_datos_destino
 10     FROM DESTINO_SOLICITUD
 11     WHERE DSO_ID_DESTINO = p_id_destino_sol;
 12   EXCEPTION
 13     WHEN NO_DATA_FOUND THEN
 14       RAISE_APPLICATION_ERROR(-20009, 'Solicitud de destino no encontrada');
 15   END;
 16
 17   -- Asignar ID para el nuevo destino
 18   SELECT seq_destino.NEXTVAL INTO p_id_destino FROM DUAL;
 19
 20   -- Crear el destino aprobado
 21   INSERT INTO DESTINO (
 22     DES_ID_DESTINO,
 23     DES_ID_USUARIO_AUTOR,
 24     DES_NOMBRE,
 25     DES_DESCRIPCION,
 26     DES_ID_PROVINCIA,
 27     DES_ID_DISTRITO,
 28     DES_UBICACION_MAPA_URL,
 29     DES_TIPO,
 30     DES_PRECIO_ENTRADA,
 31     DES_DESCRIPCION_EST,
 32     DES_HORARIO,
 33     DES_PROMEDIO_CALIFICACION
 34   ) VALUES (
 35     p_id_destino,
 36     v_datos_destino.DSO_ID_USUARIO_AUTOR,
 37     v_datos_destino.DSO_NOMBRE,
 38     v_datos_destino.DSO_DESCRIPCION,
 39     v_datos_destino.DSO_ID_PROVINCIA,
 40     v_datos_destino.DSO_ID_DISTRITO,
 41     v_datos_destino.DSO_UBICACION_MAPA_URL,
 42     v_datos_destino.DSO_TIPO,
 43     v_datos_destino.DSO_PRECIO_ENTRADA,
 44     v_datos_destino.DSO_DESCRIPCION_EST,
 45     v_datos_destino.DSO_HORARIO,
 46     v_datos_destino.DSO_PROMEDIO_CALIFICACION
 47   );
 48
 49   -- Copiar las imágenes
 50   FOR img IN (
 51     SELECT DSI_URL_IMAGEN, DSI_ORDEN
 52     FROM DESTINO_SOLICITUD IMAGEN
 53     WHERE DSI_ID_DESTINO_SOL = p_id_destino_sol
 54     ORDER BY DSI_ORDEN
 55   ) LOOP
 56     INSERT INTO DESTINO_IMAGEN (
 57       DIM_ID_IMAGEN,
 58       DIM_ID_DESTINO,
 59       DIM_URL_IMAGEN,
 60       DIM_ORDEN
 61     ) VALUES (
 62       seq_destino_img.NEXTVAL,
 63       p_id_destino,
 64       img.DSI_URL_IMAGEN,
 65       img.DSI_ORDEN
 66     );
 67   END LOOP;
 68
 69   -- Eliminar la solicitud y sus imágenes (las imágenes se eliminan automáticamente por el ON DELETE CASCADE)
 70   DELETE FROM DESTINO_SOLICITUD WHERE DSO_ID_DESTINO = p_id_destino_sol;
 71
 72   COMMIT;
 73 EXCEPTION
 74   WHEN OTHERS THEN
 75     ROLLBACK;
 76     RAISE_APPLICATION_ERROR(-20010, 'Error al aprobar destino: ' || SQLERRM);
 77 END aprobar_destino;
 78 /
```

6. Procedimiento solicitar_plan:

```
SQL> --Procedimiento para solicitar la publicación de un plan
SQL> CREATE OR REPLACE PROCEDURE solicitar_plan(
  2     p_id_empresa IN NUMBER,
  3     p_nombre IN VARCHAR2,
  4     p_descripcion IN CLOB,
  5     p_precio IN NUMBER,
  6     p_horarios IN VARCHAR2,
  7     p_fecha_inicio IN DATE DEFAULT NULL,
  8     p_fecha_final IN DATE DEFAULT NULL,
  9     p_cantidad_personas IN NUMBER,
 10     p_requisitos IN CLOB DEFAULT NULL,
 11     p_id_provincia IN NUMBER,
 12     p_id_distrito IN NUMBER,
 13     p_ubicacion_mapa IN VARCHAR2 DEFAULT NULL,
 14     p_comprar_url IN VARCHAR2 DEFAULT NULL,
 15     p_id_plan_sol OUT NUMBER
 16 ) IS
 17 BEGIN
 18     -- Validar que la empresa existe
 19     DECLARE
 20         v_empresa NUMBER;
 21     BEGIN
 22         SELECT EMP_ID_EMPRESA INTO v_empresa
 23         FROM EMPRESA
 24         WHERE EMP_ID_EMPRESA = p_id_empresa;
 25     EXCEPTION
 26         WHEN NO_DATA_FOUND THEN
 27             RAISE_APPLICATION_ERROR(-20011, 'La empresa no existe');
 28     END;
 29
 30     -- Validar fechas
 31     IF p_fecha_inicio IS NOT NULL AND p_fecha_final IS NOT NULL THEN
 32         IF p_fecha_inicio > p_fecha_final THEN
 33             RAISE_APPLICATION_ERROR(-20012, 'La fecha de inicio no puede ser posterior a la fecha final');
 34         END IF;
 35     END IF;
 36
 37     -- Asignar ID para la solicitud
 38     SELECT seq_plan_solicitud.NEXTVAL INTO p_id_plan_sol FROM DUAL;
 39
 40     -- Insertar la solicitud de plan
 41     INSERT INTO PLAN_SOLICITUD (
 42         PSO_ID_PLAN_SOLICITUD,
 43         PSO_ID_EMPRESA,
 44         PSO_NOMBRE,
 45         PSO_DESCRIPCION_PRINCIPAL,
 46         PSO_PRECIO,
 47         PSO_HORARIOS_DISPONIBLES,
 48         PSO_FECHA_INICIO,
 49         PSO_FECHA_FINAL,
 50         PSO_CANTIDAD_PERSONAS,
 51         PSO_REQUISITOS,
 52         PSO_ID_PROVINCIA,
 53         PSO_ID_DISTrito,
 54         PSO_UBICACION_MAPA_URL,
 55         PSO_COMPRAR_URL,
 56         PSO_ESTADO
 57     ) VALUES (
 58         p_id_plan_sol,
 59         p_id_empresa,
 60         p_nombre,
 61         p_descripcion,
 62         p_precio,
 63         p_horarios,
 64         p_fecha_inicio,
 65         p_fecha_final,
 66         p_cantidad_personas,
 67         p_requisitos,
 68         p_id_provincia,
 69         p_id_distrito,
 70         p_ubicacion_mapa,
 71         p_comprar_url,
 72         'PENDIENTE'
 73     );
 74
 75     COMMIT;
 76 EXCEPTION
 77     WHEN OTHERS THEN
 78         ROLLBACK;
 79         RAISE_APPLICATION_ERROR(-20013, 'Error al solicitar plan: ' || SQLERRM);
 80 END solicitar_plan;
 81 /
```


7. Procedimiento agregar_imagen_plan_sol:

```
SQL> --Procedimiento para agregar imágenes a una solicitud de plan
SQL> CREATE OR REPLACE PROCEDURE agregar_imagen_plan_sol(
  2   p_id_plan_sol IN NUMBER,
  3   p_url_imagen IN VARCHAR2,
  4   p_orden IN NUMBER DEFAULT NULL,
  5   p_id_imagen OUT NUMBER
  6 ) IS
  7   v_orden NUMBER;
  8 BEGIN
  9   -- Verificar que la solicitud existe
 10  DECLARE
 11    v_plan_sol NUMBER;
 12  BEGIN
 13    SELECT PSO_ID_PLAN_SOLICITUD INTO v_plan_sol
 14    FROM PLAN_SOLICITUD
 15    WHERE PSO_ID_PLAN_SOLICITUD = p_id_plan_sol;
 16  EXCEPTION
 17    WHEN NO_DATA_FOUND THEN
 18      RAISE_APPLICATION_ERROR(-20014, 'La solicitud de plan no existe');
 19  END;
 20
 21  -- Si no se especificó un orden, asignar el siguiente
 22  IF p_orden IS NULL THEN
 23    SELECT NVL(MAX(PSE_ORDEN), 0) + 1 INTO v_orden
 24    FROM PLAN_SOLICITUD_IMAGEN
 25    WHERE PSI_ID_PLAN_SOLICITUD = p_id_plan_sol;
 26  ELSE
 27    v_orden := p_orden;
 28  END IF;
 29
 30  -- Asignar ID para la imagen
 31  SELECT seq_plan_sol_img.NEXTVAL INTO p_id_imagen FROM DUAL;
 32
 33  -- Insertar la imagen
 34  INSERT INTO PLAN_SOLICITUD_IMAGEN (
 35    PSI_ID_IMAGEN,
 36    PSI_ID_PLAN_SOLICITUD,
 37    PSI_URL_IMAGEN,
 38    PSI_ORDEN
 39  ) VALUES (
 40    p_id_imagen,
 41    p_id_plan_sol,
 42    p_url_imagen,
 43    v_orden
 44  );
 45
 46  COMMIT;
 47 EXCEPTION
 48  WHEN OTHERS THEN
 49    ROLLBACK;
 50    RAISE_APPLICATION_ERROR(-20015, 'Error al agregar imagen: ' || SQLERRM);
 51 END agregar_imagen_plan_sol;
 52 /
```

Procedure created.

8. Procedimiento procesar_solicitud_destino:

```
Procedure created.

SQL> CREATE OR REPLACE PROCEDURE procesar_solicitud_destino(
  2     p_id_solicitud IN NUMBER,    -- ID en DESTINO_SOLICITUD
  3     p_estado       IN VARCHAR2,  -- 'APROBADO' o 'RECHAZADO'
  4     p_id_destino    OUT NUMBER    -- Nuevo ID en DESTINO; NULL si se rechaza
  5 ) IS
  6     v_cnt          NUMBER;
  7     v_sol          DESTINO_SOLICITUD%ROWTYPE;
  8 BEGIN
  9     -- Validar parámetros
 10     IF p_estado NOT IN ('APROBADO','RECHAZADO') THEN
 11         RAISE_APPLICATION_ERROR(-20030, 'Estado inválido. Use APROBADO o RECHAZADO');
 12     END IF;
 13
 14     -- Verificar existencia
 15     SELECT COUNT(*) INTO v_cnt
 16     FROM DESTINO_SOLICITUD
 17     WHERE DSO_ID_DESTINO = p_id_solicitud;
 18     IF v_cnt = 0 THEN
 19         RAISE_APPLICATION_ERROR(-20031, 'Solicitud de destino no encontrada');
 20     END IF;
 21
 22     -- Cargar datos
 23     SELECT * INTO v_sol
 24     FROM DESTINO_SOLICITUD
 25     WHERE DSO_ID_DESTINO = p_id_solicitud;
 26
 27     -- Flujos
 28     IF p_estado = 'APROBADO' THEN
 29         -- 1. Crear DESTINO
 30         SELECT seq_destino.NEXTVAL INTO p_id_destino FROM DUAL;
 31         INSERT INTO DESTINO (
 32             DES_ID_DESTINO,
 33             DES_ID_USUARIO_AUTOR,
 34             DES_NOMBRE,
 35             DES_DESCRIPCION,
 36             DES_ID_PROVINCIA,
 37             DES_ID_DISTRITO,
 38             DES_UBICACION_MAPA_URL,
 39             DES_TIPO,
 40             DES_PRECIO_ENTRADA,
 41             DES_DESCRIPCION_EST,
 42             DES_HORARIO,
 43             DES_PROMEDIO_CALIFICACION
 44         ) VALUES (
 45             p_id_destino,
 46             v_sol.DSO_ID_USUARIO_AUTOR,
 47             v_sol.DSO_NOMBRE,
 48             v_sol.DSO_DESCRIPCION,
 49             v_sol.DSO_ID_PROVINCIA,
 50             v_sol.DSO_ID_DISTRITO,
 51             v_sol.DSO_UBICACION_MAPA_URL,
 52             v_sol.DSO_TIPO,
 53             v_sol.DSO_PRECIO_ENTRADA,
 54             v_sol.DSO_DESCRIPCION_EST,
```

```

54         v_sol.DSO_DESCRIPCION_EST,
55         v_sol.DSO_HORARIO,
56         v_sol.DSO_PROMEDIO_CALIFICACION
57     );
58     -- 2. Copiar imágenes
59     FOR img IN (
60         SELECT DSI_URL_IMAGEN, DSI_ORDEN
61         FROM DESTINO_SOLICITUD_IMAGEN
62         WHERE DSI_ID_DESTINO_SOL = p_id_solicitud
63         ORDER BY DSI_ORDEN
64     ) LOOP
65         INSERT INTO DESTINO_IMAGEN (
66             DIM_ID_IMAGEN,
67             DIM_ID_DESTINO,
68             DIM_URL_IMAGEN,
69             DIM_ORDEN
70         ) VALUES (
71             seq_destino_img.NEXTVAL,
72             p_id_destino,
73             img.DSI_URL_IMAGEN,
74             img.DSI_ORDEN
75         );
76     END LOOP;
77     -- 3. Borrar solicitud (imágenes en cascade)
78     DELETE FROM DESTINO_SOLICITUD
79     WHERE DSO_ID_DESTINO = p_id_solicitud;
80
81     ELSE
82         -- Rechazo: marcar estado
83         UPDATE DESTINO_SOLICITUD
84         SET DSO_DESCRIPCION_EST = 'RECHAZADO'
85         WHERE DSO_ID_DESTINO = p_id_solicitud;
86         p_id_destino := NULL;
87     END IF;
88
89     COMMIT;
90 EXCEPTION
91     WHEN OTHERS THEN
92         ROLLBACK;
93         RAISE_APPLICATION_ERROR(-20032, 'Error DESTINO: ' || SQLERRM);
94 END procesar_solicitud_destino;
95 /

```

Procedure created.

9. Procedimiento procesar_solicitud_plan:

```
Seleccíonar Símbolo del sistema - sqlplus sys as sysdba

SQL> CREATE OR REPLACE PROCEDURE procesar_solicitud_plan(
  2     p_id_solicitud IN NUMBER,      -- ID en PLAN_SOLICITUD
  3     p_estado       IN VARCHAR2,    -- 'APROBADO' o 'RECHAZADO'
  4     p_id_plan      OUT NUMBER      -- Nuevo ID en PLAN; NULL si se rechaza
  5 ) IS
  6     v_cnt          NUMBER;
  7     v_sol          PLAN_SOLICITUD%ROWTYPE;
  8 BEGIN
  9     -- Validar parámetros
 10     IF p_estado NOT IN ('APROBADO','RECHAZADO') THEN
 11         RAISE_APPLICATION_ERROR(-20040, 'Estado inválido. Use APROBADO o RECHAZADO');
 12     END IF;
 13
 14     -- Verificar existencia
 15     SELECT COUNT(*) INTO v_cnt
 16     FROM PLAN_SOLICITUD
 17     WHERE PSO_ID_PLAN_SOLICITUD = p_id_solicitud;
 18     IF v_cnt = 0 THEN
 19         RAISE_APPLICATION_ERROR(-20041, 'Solicitud de plan no encontrada');
 20     END IF;
 21
 22     -- Cargar datos
 23     SELECT * INTO v_sol
 24     FROM PLAN_SOLICITUD
 25     WHERE PSO_ID_PLAN_SOLICITUD = p_id_solicitud;
 26
 27     -- Verificar estado actual
 28     IF v_sol.PSO_ESTADO NOT IN ('PENDIENTE','EN_REVISION') THEN
 29         RAISE_APPLICATION_ERROR(-20042, 'Ya procesada');
 30     END IF;
 31
 32     -- Flujos
 33     IF p_estado = 'APROBADO' THEN
 34         -- 1. Crear PLAN
 35         SELECT seq_plan.NEXTVAL INTO p_id_plan FROM DUAL;
 36         INSERT INTO PLAN (
 37             PLA_ID_PLAN,
 38             PLA_ID_EMPRESA,
 39             PLA_NOMBRE,
 40             PLA_DESCRIPCION_PRINCIPAL,
 41             PLA_PRECIO,
 42             PLA_HORARIOS_DISPONIBLES,
 43             PLA_FECHA_INICIO,
 44             PLA_FECHA_FINAL,
 45             PLA_CANTIDAD_PERSONAS,
 46             PLA_REQUISITOS,
 47             PLA_ID_PROVINCIA,
 48             PLA_ID_DISTRITO,
 49             PLA_UBICACION_MAPA_URL,
 50             PLA_COMPRAR_URL
```

```

50         PLA_COMPRAR_URL
51     ) VALUES (
52         p_id_plan,
53         v_sol.PSO_ID_EMPRESA,
54         v_sol.PSO_NOMBRE,
55         v_sol.PSO_DESCRIPCION_PRINCIPAL,
56         v_sol.PSO_PRECIO,
57         v_sol.PSO_HORARIOS_DISPONIBLES,
58         v_sol.PSO_FECHA_INICIO,
59         v_sol.PSO_FECHA_FINAL,
60         v_sol.PSO_CANTIDAD_PERSONAS,
61         v_sol.PSO_REQUISITOS,
62         v_sol.PSO_ID_PROVINCIA,
63         v_sol.PSO_ID_DISTRITO,
64         v_sol.PSO_UBICACION_MAPA_URL,
65         v_sol.PSO_COMPRAR_URL
66     );
67     -- 2. Copiar imágenes
68     FOR img IN (
69         SELECT PSI_URL_IMAGEN, PSI_ORDEN
70         FROM PLAN_SOLICITUD_IMAGEN
71         WHERE PSI_ID_PLAN_SOLICITUD = p_id_solicitud
72         ORDER BY PSI_ORDEN
73     ) LOOP
74         INSERT INTO PLAN_IMAGEN (
75             PIM_ID_IMAGEN,
76             PIM_ID_PLAN,
77             PIM_URL_IMAGEN,
78             PIM_ORDEN
79         ) VALUES (
80             seq_plan_img.NEXTVAL,
81             p_id_plan,
82             img.PSI_URL_IMAGEN,
83             img.PSI_ORDEN
84         );
85     END LOOP;
86     -- 3. Relación con distrito (si aplica)
87     IF v_sol.PSO_ID_DISTRITO IS NOT NULL THEN
88         INSERT INTO PLAN_DISTRITO (
89             PD_ID_PLAN,
90             PD_ID_DISTRITO
91         ) VALUES (
92             p_id_plan,
93             v_sol.PSO_ID_DISTRITO
94         );
95     END IF;
96     -- 4. Marcar aprobado y fecha
97     UPDATE PLAN_SOLICITUD
98         SET PSO_ESTADO = 'APROBADO',
99             PSO_FECHA_REVISION = SYSTIMESTAMP
100     WHERE PSO_ID_PLAN_SOLICITUD = p_id_solicitud;
101
102     ELSE
103         -- Rechazo
104         UPDATE PLAN_SOLICITUD
105             SET PSO_ESTADO = 'RECHAZADO',
106                 PSO_FECHA_REVISION = SYSTIMESTAMP
107         WHERE PSO_ID_PLAN_SOLICITUD = p_id_solicitud;
108         p_id_plan := NULL;
109     END IF;
110
111     COMMIT;
112 EXCEPTION
113     WHEN OTHERS THEN
114         ROLLBACK;
115         RAISE_APPLICATION_ERROR(-20043, 'Error PLAN: ' || SQLERRM);
116 END procesar_solicitud_plan;
117 /

```

10. Procedimiento gestionar_favorito

```
Procedure created.

SQL> --Procedimiento para agregar/quitar un destino a favoritos
SQL> CREATE OR REPLACE PROCEDURE gestionar_favorito(
  2     p_id_usuario IN NUMBER,
  3     p_id_destino IN NUMBER,
  4     p_accion IN VARCHAR2 -- 'AGREGAR' o 'QUITAR'
  5 ) IS
  6     v_existe NUMBER;
  7 BEGIN
  8     -- Validar usuario
  9     DECLARE
 10         v_usuario NUMBER;
 11     BEGIN
 12         SELECT USU_ID_USUARIO INTO v_usuario
 13         FROM USUARIO
 14         WHERE USU_ID_USUARIO = p_id_usuario;
 15     EXCEPTION
 16         WHEN NO_DATA_FOUND THEN
 17             RAISE_APPLICATION_ERROR(-20042, 'El usuario no existe');
 18     END;
 19
 20     -- Validar destino
 21     DECLARE
 22         v_destino NUMBER;
 23     BEGIN
 24         SELECT DES_ID_DESTINO INTO v_destino
 25         FROM DESTINO
 26         WHERE DES_ID_DESTINO = p_id_destino;
 27     EXCEPTION
 28         WHEN NO_DATA_FOUND THEN
 29             RAISE_APPLICATION_ERROR(-20043, 'El destino no existe');
 30     END;
 31
 32     -- Validar acción
 33     IF p_accion NOT IN ('AGREGAR', 'QUITAR') THEN
 34         RAISE_APPLICATION_ERROR(-20044, 'Acción inválida. Use AGREGAR o QUITAR');
 35     END IF;
 36
 37     -- Verificar si ya existe el favorito
 38     SELECT COUNT(*) INTO v_existe
 39     FROM FAVORITO_DESTINO
 40     WHERE FAV_ID_USUARIO = p_id_usuario
 41           AND FAV_ID_DESTINO = p_id_destino;
 42
 43     -- Verificar si ya existe el favorito
 44     SELECT COUNT(*) INTO v_existe
 45     FROM FAVORITO_DESTINO
 46     WHERE FAV_ID_USUARIO = p_id_usuario
 47           AND FAV_ID_DESTINO = p_id_destino;
 48
 49     -- Realizar la acción correspondiente
 50     IF p_accion = 'AGREGAR' THEN
 51         IF v_existe = 0 THEN
 52             INSERT INTO FAVORITO_DESTINO (
 53                 FAV_ID_USUARIO,
 54                 FAV_ID_DESTINO,
 55                 FAV_FECHA_AGREGADO
 56             ) VALUES (
 57                 p_id_usuario,
 58                 p_id_destino,
 59                 SYSTIMESTAMP
 60             );
 61         END IF;
 62     ELSIF p_accion = 'QUITAR' THEN
 63         IF v_existe > 0 THEN
 64             DELETE FROM FAVORITO_DESTINO
 65             WHERE FAV_ID_USUARIO = p_id_usuario
 66                   AND FAV_ID_DESTINO = p_id_destino;
 67         END IF;
 68     END IF;
 69
 70     COMMIT;
 71 EXCEPTION
 72     WHEN OTHERS THEN
 73         ROLLBACK;
 74         RAISE_APPLICATION_ERROR(-20045, 'Error al gestionar favorito: ' || SQLERRM);
 75 END gestionar_favorito;
 76 /

Procedure created.
```

11. Funciones contar_destinos_pendientes y contar_planes_destinos.

```
SQL> CREATE OR REPLACE FUNCTION contar_destinos_pendientes RETURN NUMBER IS
  2     total NUMBER;
  3 BEGIN
  4     SELECT COUNT(*)
  5     INTO total
  6     FROM DESTINO_SOLICITUD
  7     WHERE DSO_DESCRIPCION_EST = 'PENDIENTE';
  8
  9     RETURN total;
 10 END;
 11
 12 /
```

Function created.

Procedure created.

```
SQL> -- Contar planes pendientes
SQL> CREATE OR REPLACE FUNCTION contar_planes_pendientes RETURN NUMBER IS
  2     total NUMBER;
  3 BEGIN
  4     SELECT COUNT(*) INTO total FROM PLAN_SOLICITUD WHERE PSO_ESTADO = 'PENDIENTE';
  5     RETURN total;
  6 END;
  7 /
```

12. Trigger trg_validar_plan:

```
SQL>
SQL> --Trigger para validar la coherencia de datos en PLAN
SQL> CREATE OR REPLACE TRIGGER trg_validar_plan
  2 BEFORE INSERT OR UPDATE ON PLAN
  3 FOR EACH ROW
  4 BEGIN
  5     -- Validar precio
  6     IF :NEW.PLA_PRECIO < 0 THEN
  7         RAISE_APPLICATION_ERROR(-20028, 'El precio no puede ser negativo');
  8     END IF;
  9
 10     -- Validar fechas
 11     IF :NEW.PLA_FECHA_INICIO IS NOT NULL AND :NEW.PLA_FECHA_FINAL IS NOT NULL THEN
 12         IF :NEW.PLA_FECHA_INICIO > :NEW.PLA_FECHA_FINAL THEN
 13             RAISE_APPLICATION_ERROR(-20029, 'La fecha de inicio no puede ser posterior a la fecha final');
 14         END IF;
 15     END IF;
 16
 17     -- Validar cantidad de personas
 18     IF :NEW.PLA_CANTIDAD_PERSONAS <= 0 THEN
 19         RAISE_APPLICATION_ERROR(-20030, 'La cantidad de personas debe ser mayor a cero');
 20     END IF;
 21
 22     -- Validar provincia y distrito
 23     IF :NEW.PLA_ID_PROVINCIA IS NOT NULL AND :NEW.PLA_ID_DISTRITO IS NOT NULL THEN
 24         -- Verificar que el distrito pertenece a la provincia
 25         DECLARE
 26             v_provincia NUMBER;
 27         BEGIN
 28             SELECT DIS_ID_PROVINCIA INTO v_provincia
 29             FROM DISTRITO
 30             WHERE DIS_ID_DISTRITO = :NEW.PLA_ID_DISTRITO;
 31
 32             IF v_provincia != :NEW.PLA_ID_PROVINCIA THEN
 33                 RAISE_APPLICATION_ERROR(-20031, 'El distrito no pertenece a la provincia especificada');
 34             END IF;
 35         EXCEPTION
 36             WHEN NO_DATA_FOUND THEN
 37                 RAISE_APPLICATION_ERROR(-20032, 'El distrito especificado no existe');
 38         END;
 39     END IF;
 40 END;
 41 /
```


13. Trigger trg_validar_destino:

```
Trigger created.

SQL>
SQL>
SQL> --Trigger similar para DESTINO
SQL> CREATE OR REPLACE TRIGGER trg_validar_destino
  2 BEFORE INSERT OR UPDATE ON DESTINO
  3 FOR EACH ROW
  4 BEGIN
  5     -- Validar precio de entrada
  6     IF :NEW.DES_PRECIO_ENTRADA < 0 THEN
  7         RAISE_APPLICATION_ERROR(-20033, 'El precio de entrada no puede ser negativo');
  8     END IF;
  9
 10    -- Validar provincia y distrito
 11    IF :NEW.DES_ID_PROVINCIA IS NOT NULL AND :NEW.DES_ID_DISTRITO IS NOT NULL THEN
 12        -- Verificar que el distrito pertenece a la provincia
 13        DECLARE
 14            v_provincia NUMBER;
 15        BEGIN
 16            SELECT DIS_ID_PROVINCIA INTO v_provincia
 17            FROM DISTRITO
 18            WHERE DIS_ID_DISTRITO = :NEW.DES_ID_DISTRITO;
 19
 20            IF v_provincia != :NEW.DES_ID_PROVINCIA THEN
 21                RAISE_APPLICATION_ERROR(-20034, 'El distrito no pertenece a la provincia especificada');
 22            END IF;
 23        EXCEPTION
 24            WHEN NO_DATA_FOUND THEN
 25                RAISE_APPLICATION_ERROR(-20035, 'El distrito especificado no existe');
 26        END;
 27    END IF;
 28
 29    -- Validar promedio de calificación
 30    IF :NEW.DES_PROMEDIO_CALIFICACION IS NOT NULL THEN
 31        IF :NEW.DES_PROMEDIO_CALIFICACION < 0 OR :NEW.DES_PROMEDIO_CALIFICACION > 5 THEN
 32            RAISE_APPLICATION_ERROR(-20036, 'El promedio de calificación debe estar entre 0 y 5');
 33        END IF;
 34    END IF;
 35 END;
 36 /

Trigger created.

SQL>
```

14. Vista de Destinos Completos

```
SQL> -- 1. Vista de destinos completos con ubicación
SQL> CREATE OR REPLACE VIEW V_DESTINOS_COMPLETOS AS
 2  SELECT
 3      d.DES_ID_DESTINO,
 4      d.DES_NOMBRE,
 5      d.DES_DESCRIPCION,
 6      d.DES_TIPO,
 7      d.DES_PRECIO_ENTRADA,
 8      d.DES_HORARIO,
 9      d.DES_PROMEDIO_CALIFICACION,
10      p.PRO_NOMBRE AS PROVINCIA,
11      dt.DIS_NOMBRE AS DISTRITO,
12      d.DES_UBICACION_MAPA_URL,
13      u.USU_NOMBRE || ' ' || u.USU_APELLIDO_P AS AUTOR,
14      d.DES_FECHA_PUBLICACION
15  FROM DESTINO d
16  JOIN USUARIO u ON d.DES_ID_USUARIO_AUTOR = u.USU_ID_USUARIO
17  LEFT JOIN PROVINCIA p ON d.DES_ID_PROVINCIA = p.PRO_ID_PROVINCIA
18  LEFT JOIN DISTRITO dt ON d.DES_ID_DISTRITO = dt.DIS_ID_DISTRITO;
```

15. Vista Planes Disponibles

```
SQL> -- 2. Vista de planes turísticos con información de empresa
SQL> CREATE OR REPLACE VIEW V_PLANES_DISPONIBLES AS
 2  SELECT
 3      p.PLA_ID_PLAN,
 4      p.PLA_NOMBRE,
 5      p.PLA_DESCRIPCION_PRINCIPAL,
 6      p.PLA_PRECIO,
 7      p.PLA_HORARIOS_DISPONIBLES,
 8      p.PLA_FECHA_INICIO,
 9      p.PLA_FECHA_FINAL,
10      p.PLA_CANTIDAD_PERSONAS,
11      p.PLA_PROMEDIO_CALIFICACION,
12      e.EMP_NOMBRE AS EMPRESA,
13      e.EMP_TELEFONO,
14      e.EMP_CORREO AS EMPRESA_CORREO,
15      pr.PRO_NOMBRE AS PROVINCIA,
16      d.DIS_NOMBRE AS DISTRITO,
17      p.PLA_COMPRAR_URL
18  FROM PLAN p
19  JOIN EMPRESA e ON p.PLA_ID_EMPRESA = e.EMP_ID_EMPRESA
20  LEFT JOIN PROVINCIA pr ON p.PLA_ID_PROVINCIA = pr.PRO_ID_PROVINCIA
21  LEFT JOIN DISTRITO d ON p.PLA_ID_DISTRITO = d.DIS_ID_DISTRITO
22  WHERE p.PLA_FECHA_FINAL >= SYSDATE OR p.PLA_FECHA_FINAL IS NULL;

View created.

SQL> /
```

16. Vista Actividades Recientes

```
SQL> CREATE OR REPLACE VIEW V_ACTIVIDAD_RECIENTE AS
 2  SELECT
 3      'DESTINO_PUBLICADO' AS TIPO_ACTIVIDAD,
 4      d.DES_NOMBRE AS NOMBRE,
 5      u.USU_NOMBRE || ' ' || u.USU_APELLIDO_P AS USUARIO,
 6      d.DES_FECHA_PUBLICACION AS FECHA_ACTIVIDAD,
 7      'Usuario publicó nuevo destino' AS DESCRIPCION
 8  FROM DESTINO d
 9  JOIN USUARIO u ON d.DES_ID_USUARIO_AUTOR = u.USU_ID_USUARIO
10  WHERE d.DES_FECHA_PUBLICACION >= SYSDATE - 30
11  UNION ALL
12  SELECT
13      'PLAN_PUBLICADO' AS TIPO_ACTIVIDAD,
14      p.PLA_NOMBRE AS NOMBRE,
15      e.EMP_NOMBRE AS USUARIO,
16      p.PLA_FECHA_PUBLICACION AS FECHA_ACTIVIDAD,
17      'Empresa publicó nuevo plan' AS DESCRIPCION
18  FROM PLAN p
19  JOIN EMPRESA e ON p.PLA_ID_EMPRESA = e.EMP_ID_EMPRESA
20  WHERE p.PLA_FECHA_PUBLICACION >= SYSDATE - 30
21  UNION ALL
22  SELECT
23      'SOLICITUD_DESTINO' AS TIPO_ACTIVIDAD,
24      ds.DSO_NOMBRE AS NOMBRE,
25      u.USU_NOMBRE || ' ' || u.USU_APELLIDO_P AS USUARIO,
26      ds.DSO_FECHA_CREACION AS FECHA_ACTIVIDAD,
27      'Usuario envió solicitud de destino' AS DESCRIPCION
28  FROM DESTINO_SOLICITUD ds
29  JOIN USUARIO u ON ds.DSO_ID_USUARIO_AUTOR = u.USU_ID_USUARIO
30  WHERE ds.DSO_FECHA_CREACION >= SYSDATE - 30
31  UNION ALL
32  SELECT
33      'SOLICITUD_PLAN' AS TIPO_ACTIVIDAD,
34      ps.PSO_NOMBRE AS NOMBRE,
35      e.EMP_NOMBRE AS USUARIO,
36      ps.PSO_FECHA_SOLICITUD AS FECHA_ACTIVIDAD,
37      'Empresa envió solicitud de plan' AS DESCRIPCION
38  FROM PLAN_SOLICITUD ps
39  JOIN EMPRESA e ON ps.PSO_ID_EMPRESA = e.EMP_ID_EMPRESA
40  WHERE ps.PSO_FECHA_SOLICITUD >= SYSDATE - 30
41  ;

View created.
```

17. Vista de solicitudes Pendientes

```
SQL> CREATE OR REPLACE FORCE VIEW "V_SOLICITUDES_PENDIENTES" (  
2   "TIPO_SOLICITUD",  
3   "ID_SOLICITUD",  
4   "NOMBRE",  
5   "DESCRIPCION",  
6   "ESTADO",  
7   "FECHA_SOLICITUD",  
8   "SOLICITANTE",  
9   "CORREO_SOLICITANTE",  
10  "EMPRESA"  
11 ) AS  
12 SELECT  
13   'DESTINO' AS TIPO_SOLICITUD,  
14   ds.DSO_ID_DESTINO AS ID_SOLICITUD,  
15   ds.DSO_NOMBRE AS NOMBRE,  
16   ds.DSO_DESCRIPCION AS DESCRIPCION,  
17   ds.DSO_DESCRIPCION_EST AS ESTADO,  
18   ds.DSO_FECHA_CREACION AS FECHA_SOLICITUD,  
19   u.USU_NOMBRE || ' ' || u.USU_APELLIDO_P AS SOLICITANTE,  
20   u.USU_CORREO AS CORREO_SOLICITANTE,  
21   NULL AS EMPRESA  
22 FROM DESTINO_SOLICITUD ds  
23 JOIN USUARIO u ON ds.DSO_ID_USUARIO_AUTOR = u.USU_ID_USUARIO  
24 WHERE ds.DSO_DESCRIPCION_EST = 'PENDIENTE'  
25 UNION ALL  
26 SELECT  
27   'PLAN' AS TIPO_SOLICITUD,  
28   ps.PSO_ID_PLAN_SOLICITUD AS ID_SOLICITUD,  
29   ps.PSO_NOMBRE AS NOMBRE,  
30   ps.PSO_DESCRIPCION_PRINCIPAL AS DESCRIPCION,  
31   ps.PSO_ESTADO AS ESTADO,  
32   ps.PSO_FECHA_SOLICITUD AS FECHA_SOLICITUD,  
33   e.EMP_NOMBRE AS SOLICITANTE,  
34   e.EMP_CORREO AS CORREO_SOLICITANTE,  
35   e.EMP_NOMBRE AS EMPRESA  
36 FROM PLAN_SOLICITUD ps  
37 JOIN EMPRESA e ON ps.PSO_ID_EMPRESA = e.EMP_ID_EMPRESA  
38 WHERE ps.PSO_ESTADO = 'PENDIENTE'  
39 ;  
  
View created.  
  
SQL>
```

18. Vista Favorito Usuarios

```
SQL> CREATE OR REPLACE VIEW V_FAVORITOS_USUARIO AS
 2  SELECT
 3      u.USU_ID_USUARIO,
 4      u.USU_NOMBRE || ' ' || u.USU_APELLIDO_P AS USUARIO,
 5      d.DES_ID_DESTINO,
 6      d.DES_NOMBRE AS DESTINO,
 7      d.DES_TIPO,
 8      d.DES_PROMEDIO_CALIFICACION,
 9      p.PRO_NOMBRE AS PROVINCIA,
10      dt.DIS_NOMBRE AS DISTRITO,
11      f.FAV_FECHA_AGREGADO
12  FROM FAVORITO_DESTINO f
13  JOIN USUARIO u ON f.FAV_ID_USUARIO = u.USU_ID_USUARIO
14  JOIN DESTINO d ON f.FAV_ID_DESTINO = d.DES_ID_DESTINO
15  LEFT JOIN PROVINCIA p ON d.DES_ID_PROVINCIA = p.PRO_ID_PROVINCIA
16  LEFT JOIN DISTRITO dt ON d.DES_ID_DISTRITO = dt.DIS_ID_DISTRITO;

View created.
```

19. Vista Empresas Activas

```
SQL> CREATE OR REPLACE VIEW V_EMPRESAS_ACTIVAS AS
 2  SELECT
 3      e.EMP_ID_EMPRESA,
 4      e.EMP_NOMBRE,
 5      e.EMP_TIPO_EMPRESA,
 6      e.EMP_TELEFONO,
 7      e.EMP_CORREO,
 8      e.EMP_INSTAGRAM_URL,
 9      e.EMP_FACEBOOK_URL,
10      e.EMP_PAGINA_WEB_URL,
11      COUNT(p.PLA_ID_PLAN) AS PLANES_ACTIVOS,
12      ROUND(AVG(p.PLA_PRECIO), 2) AS PRECIO_PROMEDIO,
13      ROUND(AVG(p.PLA_PROMEDIO_CALIFICACION), 2) AS CALIFICACION_PROMEDIO
14  FROM EMPRESA e
15  LEFT JOIN PLAN p ON e.EMP_ID_EMPRESA = p.PLA_ID_EMPRESA
16      AND (p.PLA_FECHA_FINAL >= SYSDATE OR p.PLA_FECHA_FINAL IS NULL)
17  GROUP BY e.EMP_ID_EMPRESA, e.EMP_NOMBRE, e.EMP_TIPO_EMPRESA,
18      e.EMP_TELEFONO, e.EMP_CORREO, e.EMP_INSTAGRAM_URL,
19      e.EMP_FACEBOOK_URL, e.EMP_PAGINA_WEB_URL;

View created.

SQL> /
```

Conclusión

Desarrollar el proyecto Tourlstmó fue una experiencia transformadora para nosotros como estudiantes y futuros profesionales en el área de ingeniería de software. Más allá de cumplir con los requisitos académicos del curso, este proyecto nos permitió vivir de forma directa lo que implica diseñar, construir e implementar una solución tecnológica con aplicación en un contexto real. Desde el principio, supimos que queríamos ir más allá de lo básico y crear algo que no solo funcionara, sino que también fuera útil, escalable y digno de presentar en cualquier escenario profesional.

Durante el proceso, pusimos en práctica todo lo aprendido en el diseño de bases de datos, desde la creación del modelo conceptual hasta su implementación física en Oracle. Aprendimos a normalizar correctamente nuestras estructuras, a definir relaciones con integridad referencial, y a estructurar nuestras tablas pensando tanto en la eficiencia como en la claridad. La implementación de procedimientos, funciones, triggers y cursores en PL/SQL fue todo un reto, pero también un momento clave para consolidar nuestros conocimientos. Tuvimos que pensar en la lógica detrás de cada acción, en cómo manejar errores, controlar flujos de información y automatizar procesos críticos del sistema. Fue ahí donde realmente entendimos el valor de la lógica almacenada como el corazón de un sistema transaccional.

Más allá de lo técnico, este proyecto nos enseñó a trabajar como equipo. Aprendimos a dividir tareas de forma equilibrada, a escucharnos, a proponer ideas y a resolver diferencias con respeto y responsabilidad. Hubo momentos en los que las cosas no salían como esperábamos, pero supimos adaptarnos, reorganizarnos y seguir adelante. Cada integrante aportó desde sus fortalezas y conocimientos, y eso nos permitió construir una solución mucho más completa y sólida. Fue un proceso que nos hizo crecer no solo como desarrolladores, sino también como colaboradores, algo que sabemos es fundamental en cualquier entorno laboral.

Hoy podemos decir con orgullo que Tourlstmó no es solo un proyecto de curso, sino un sistema funcional, bien documentado y con potencial real para implementarse en el sector turístico panameño. Logramos integrar funciones como la gestión de destinos, planes turísticos, favoritos, autenticación y recomendaciones, todo sobre una base de

datos bien estructurada y preparada para escalar. Sabemos que aún hay espacio para mejorar y expandir esta plataforma, pero estamos convencidos de que sentamos una base sólida, profesional y de calidad.

Para nuestro grupo, Tourlstm fue mucho más que una entrega académica: fue una oportunidad de aplicar todo lo aprendido, de enfrentarnos a problemas reales y de demostrarnos a nosotros mismos que somos capaces de crear soluciones útiles y con valor. Nos llevamos no solo un sistema bien hecho, sino también una experiencia que nos preparó mejor para los desafíos que vendrán en nuestra carrera profesional.

Bibliografía/Referencias

- Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson Education.
- Coronel, C., & Morris, S. (2018). *Database Systems: Design, Implementation, & Management* (13th ed.). Cengage Learning.
- Connolly, T., & Begg, C. (2015). *Database Systems: A Practical Approach to Design, Implementation, and Management* (6th ed.). Pearson.
- Minelli, M., Chambers, M., & Dhiraj, A. (2013). *Big Data, Big Analytics: Emerging Business Intelligence and Analytic Trends for Today's Businesses*. Wiley.
- Buhalis, D., & Law, R. (2008). Progress in information technology and tourism management: 20 years on and 10 years after the Internet. *Tourism Management*, 29(4), 609-623
<https://www.sciencedirect.com/science/article/abs/pii/S0160738306000247?via%3Dihub>
- Pan, B., & Fesenmaier, D. R. (2006). Online information search: Vacation planning process. *Annals of Tourism Research*, 33(3), 809-832.
<https://www.sciencedirect.com/science/article/abs/pii/S0160738306000247?via%3Dihub>
- Oracle. (2023). *Oracle® Database PL/SQL Language Reference 19c*. Oracle Corporation.
<https://docs.oracle.com/en/database/oracle/oracle-database/19/lnpls/index.html>